Machine Learning

Deep learning: Further topics

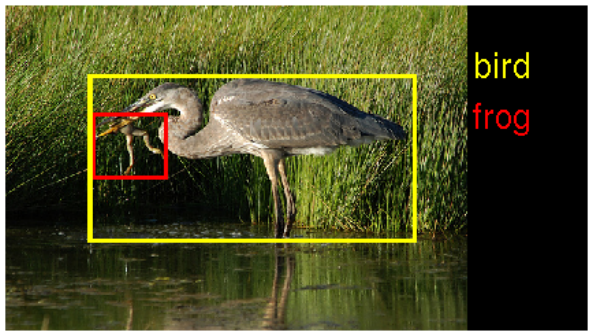
Sebastian Engelke

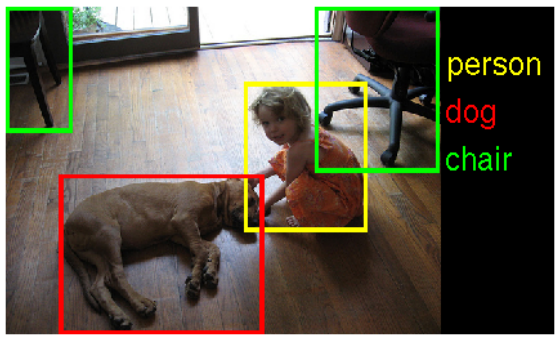
Master in Business Analytics

UNIVERSITÉ DE GENÈVE

ImageNet

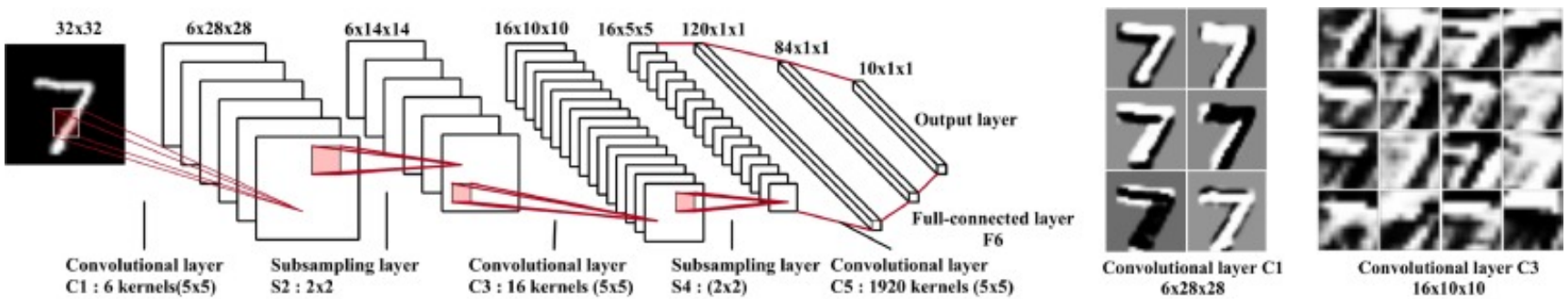
- A more complex benchmark for computer vision is the ImageNet data base: http://www.image-net.org/challenges/LSVRC/2014/
- There are 200 different categories and multiple objects can be in one image.
- Over 450k images as training and 40k images as test.
- The sizes can be different with an average image size of 482×415 pixels.





Popular network architectures

- Convolutional neural networks were first developed and used in the 1990's by Yann LeCun and others.
- The well-known LeNet architecture was successfully used to read handwritten zip codes.
- Since then, uncountable new architectures have been proposed, continuously improving the scores on the benchmarks like ImageNet.



(a) LeNet-5 network (b) Learned features

Image from https://www.sciencedirect.com/science/article/pii/S0031320317304120

AlexNet

- AlexNet first popularized convolutional neural networks in computer vision, and that systematically uses ReLU instead of sigmoid activations.
- It was submitted by Alex Krizhevsky, Ilya Sutskever and Geoff Hinton to the ImageNet ILSVRC challenge in 2012.
- It improved the then best error of 26% to 16%.
- The architecture is similar to LeNet, but was deeper, bigger, and had convolutional layers stacked on top of each other (previously, one used only a single conv. layer immediately followed by pooling).

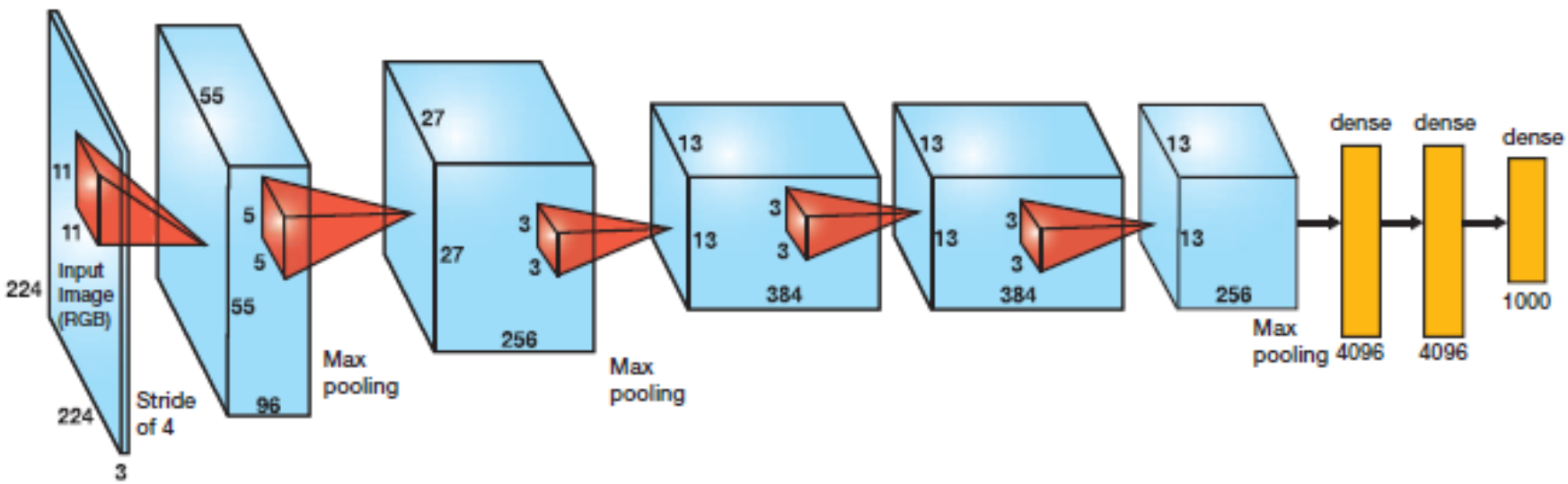


Image from http://cv-tricks.com/cnn/understand-resnet-alexnet-vgg-inception/

VGGNet

- The winner of the ILSVRC 2014 challenge was the VGGNet by Karen Simonyan and Andrew Zisserman.
- It proved that the depth of the network is critical for good performance.
- Their final model has a very homogeneous architecture with 16 layers that only performs 3×3 convolutions and 2×2 pooling from the beginning to the end.
- A downside of the VGGNet is that it is uses a lot more memory and parameters (140M).

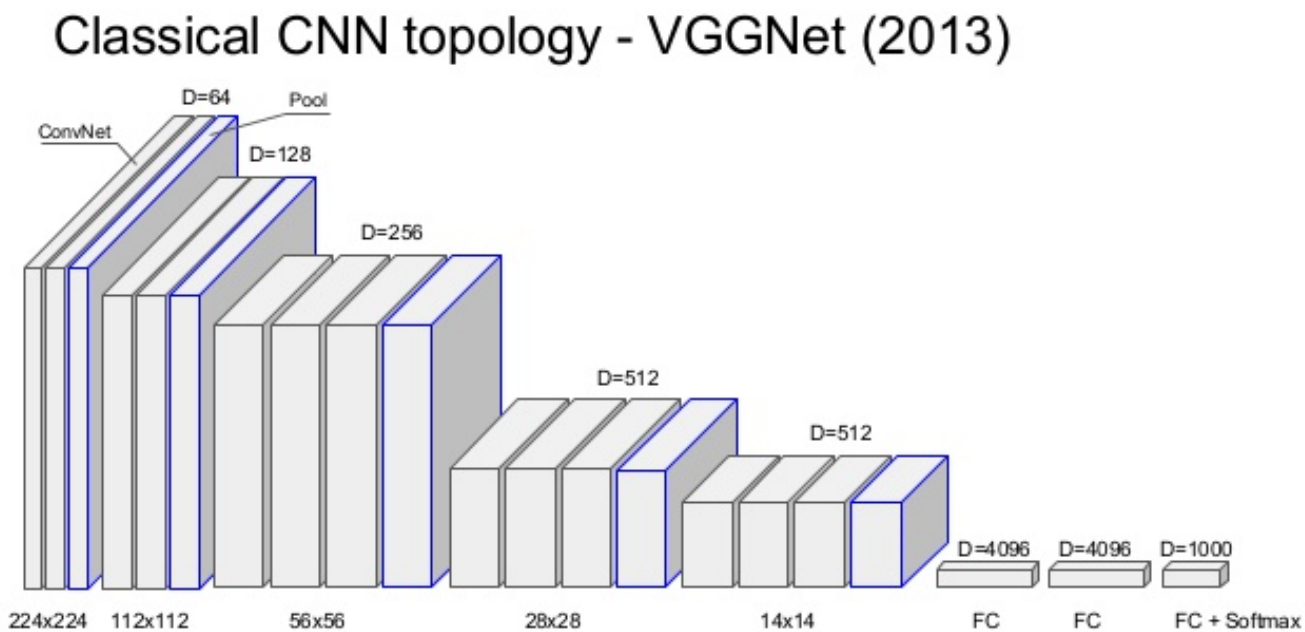
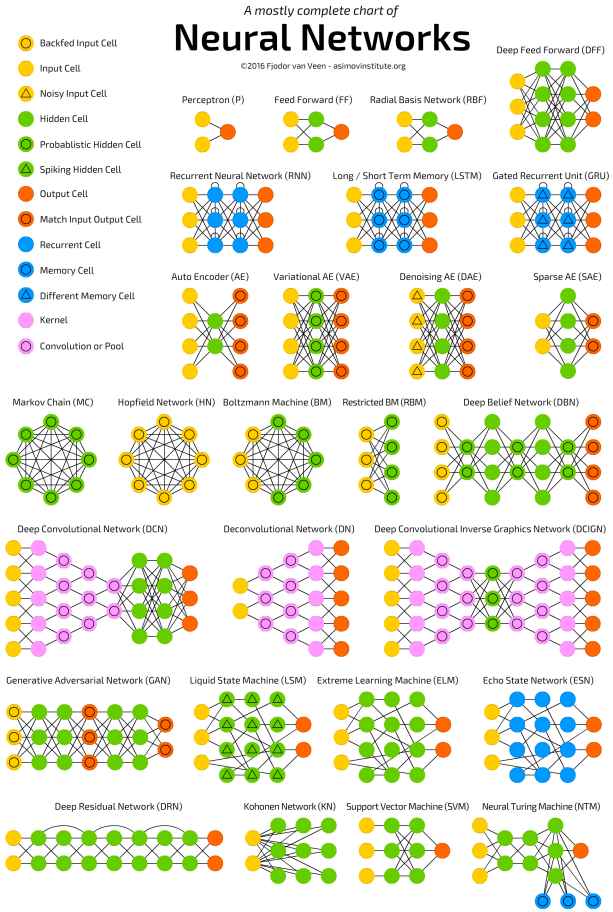


Image from http://srdas.github.io/DLBook/ConvNets.html

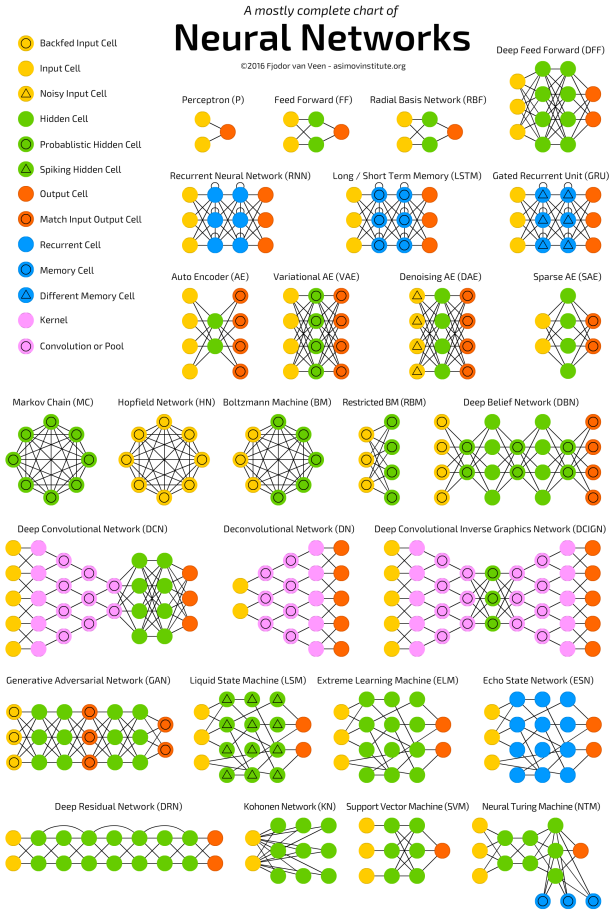
More networks

- Besides these, there are many other variations of these networks and different types of architectures (e.g., `ResNet` or Google's `Inception`).
- The variety is due to different applications, and to `active research` and advances in this area.



More networks

- Besides these, there are many other variations of these networks and different types of architectures (e.g., `ResNet` or Google's `Inception`).
- The variety is due to different applications, and to `active research` and advances in this area.



- For an overview see `https://github.com/BVLC/caffe/wiki/Model-Zoo`

Typical filter weights of first layer

- The filters of the first layer can be `interpreted` since they operate directly on the image.
- They extract `generic features` and can be seen as `edge or blob detectors`, for instance.

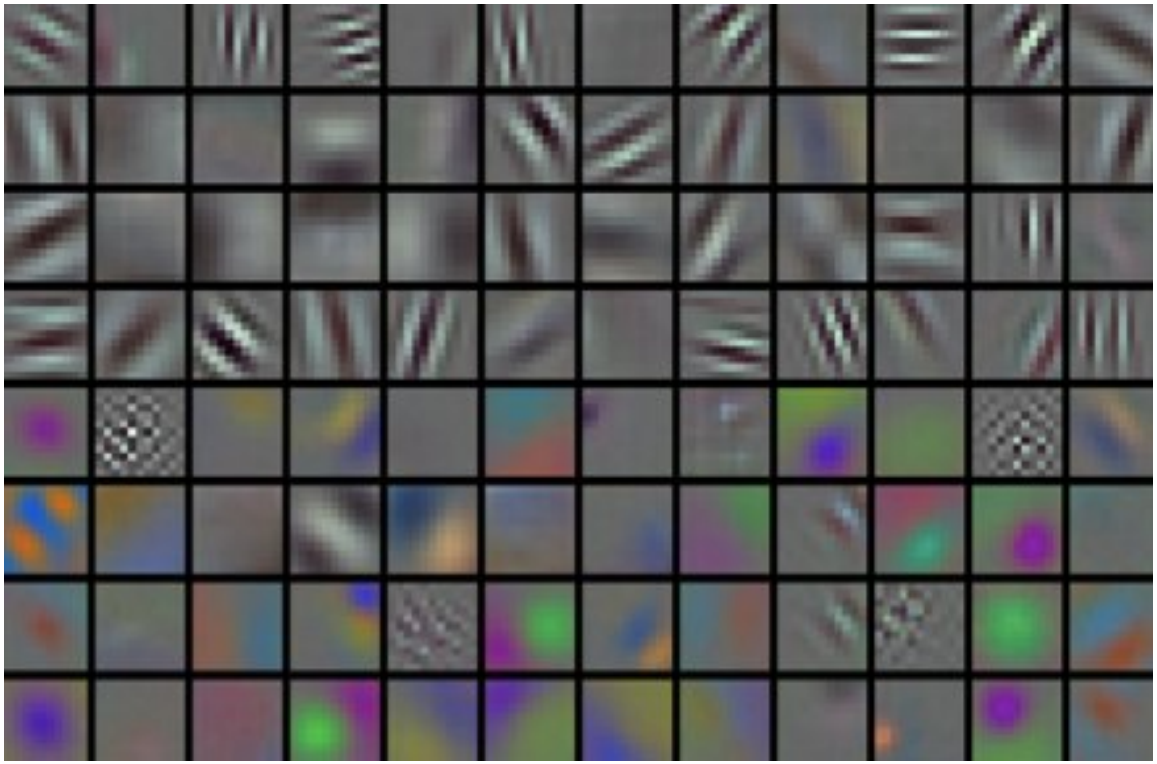


Image from `http://cs231n.github.io/neural-networks-3/`

Unconverged filters

- Sometimes the filters do not look as nice as on the previous slide, and they will not extract clear features.
- There can be different reasons for this.
- The network might not be trained sufficiently.
- The learning rate might be chosen poorly, or very low weight regularization is used.

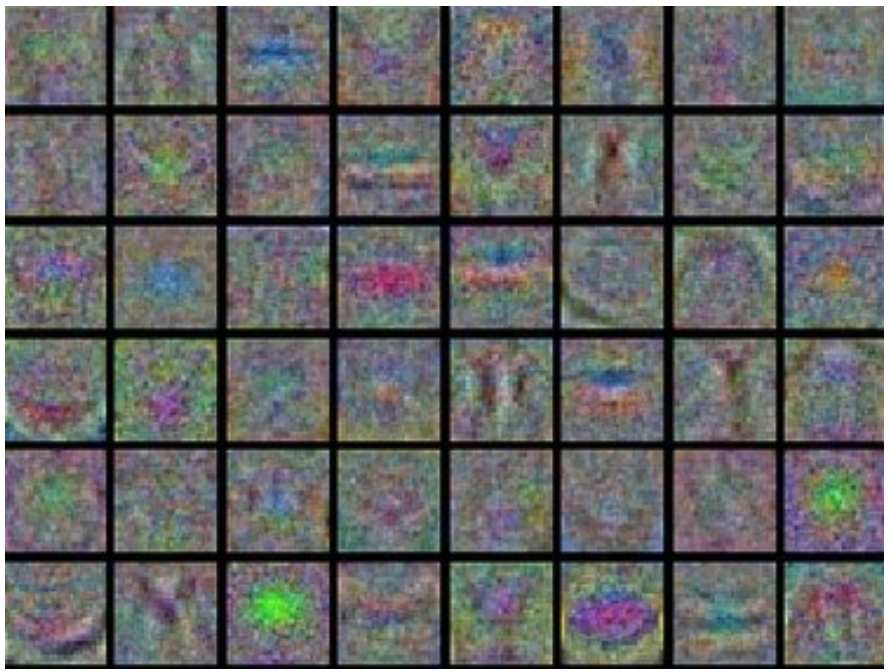


Image from http://cs231n.github.io/neural-networks-3/

Activations of deeper layers

- The filters of later, deeper layers are more `specific to the data set` used during training.
- The `task of a specific` neuron can be visualized by plotting input images that `maximally activate` that neuron.
- Below are examples for six neurons in the 5*th* (last) pooling of `AlexNet`.

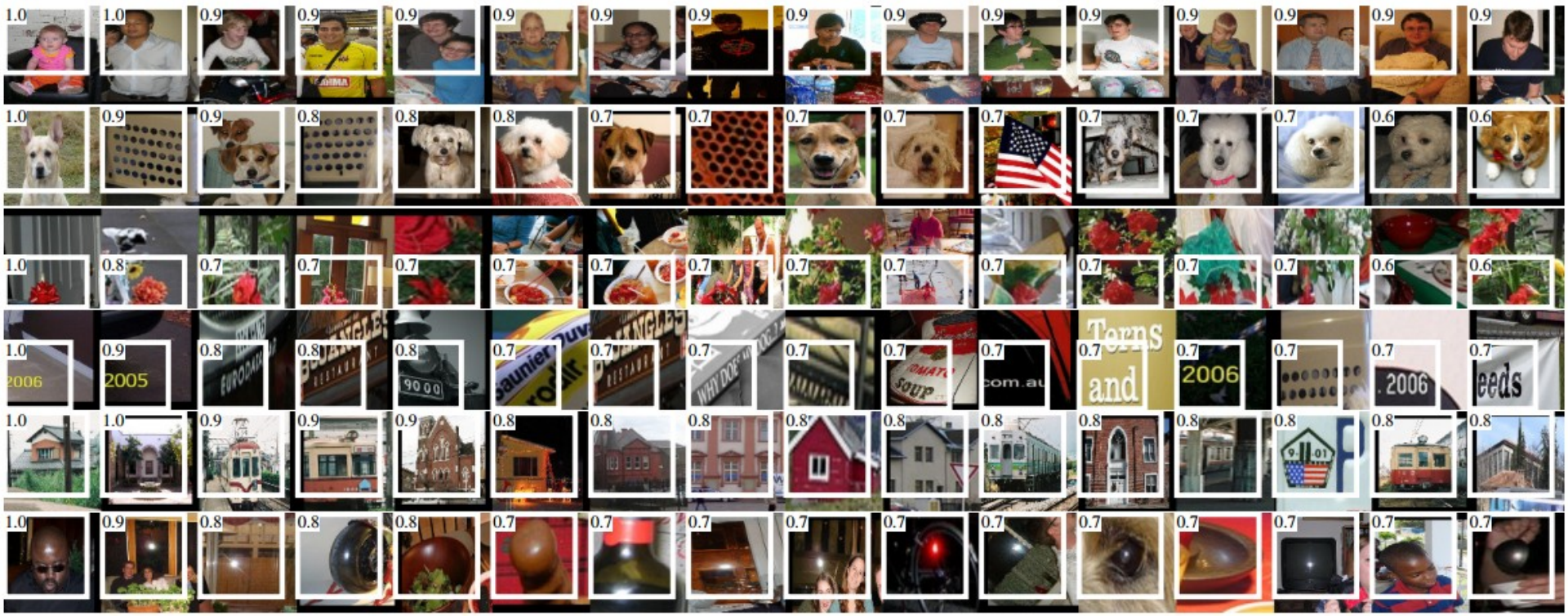


Image from Girshick et al. (2013, `https://arxiv.org/abs/1311.2524`)

Pre-trained networks

- As an end-user, you might neither have the `data` nor the `computational resources` to fit a modern network yourself `from scratch`.
- The expensive part is the `fitting`, not the `forward-pass`.

Pre-trained networks

- As an end-user, you might neither have the data nor the computational resources to fit a modern network yourself from scratch.
- The expensive part is the fitting, not the forward-pass.
- As an alternative you can use a state-of-the-art network that has been pre-trained on large data sets such as ImageNet.
- The weights for many different networks are available, e.g., `https://github.com/BVLC/caffe/wiki/Model-Zoo`

Pre-trained networks

- As an end-user, you might neither have the data nor the computational resources to fit a modern network yourself from scratch.
- The expensive part is the fitting, not the forward-pass.
- As an alternative you can use a state-of-the-art network that has been pre-trained on large data sets such as ImageNet.
- The weights for many different networks are available, e.g., https://github.com/BVLC/caffe/wiki/Model-Zoo
- This will only allow you to perform exactly the same task on your data (i.e., same input size, same output classes) as the network was trained to do.
- The input image size is not a problem, since images can typically be scaled to have the desired format.

Transfer learning

- A main issue with pre-trained networks is that they can only predict their `pre-defined categories`.
- Usually, your problem will require to predict `other categories`, e.g., flower species, cancer types, car brands, traffic signs, etc.

Transfer learning

- A main issue with pre-trained networks is that they can only predict their `pre-defined categories`.
- Usually, your problem will require to predict `other categories`, e.g., flower species, cancer types, car brands, traffic signs, etc.
- Moreover, you might only have a `small number` of training samples, certainly not enough to train a complicated CNN.

Transfer learning

- A main issue with pre-trained networks is that they can only predict their `pre-defined categories`.
- Usually, your problem will require to predict `other categories`, e.g., flower species, cancer types, car brands, traffic signs, etc.
- Moreover, you might only have a `small number` of training samples, certainly not enough to train a complicated CNN.
- `Transfer learning` is the idea to build on information from a pre-trained model, but adapt this model to your problem.
- You might for instace use a model trained on `ImageNet` to build a model that distinguishes different types of galaxies.

Transfer learning

- How can this work?
- You should see a CNN as consisting of two parts: the `generic feature extractor` and the `problem specific classifier`.

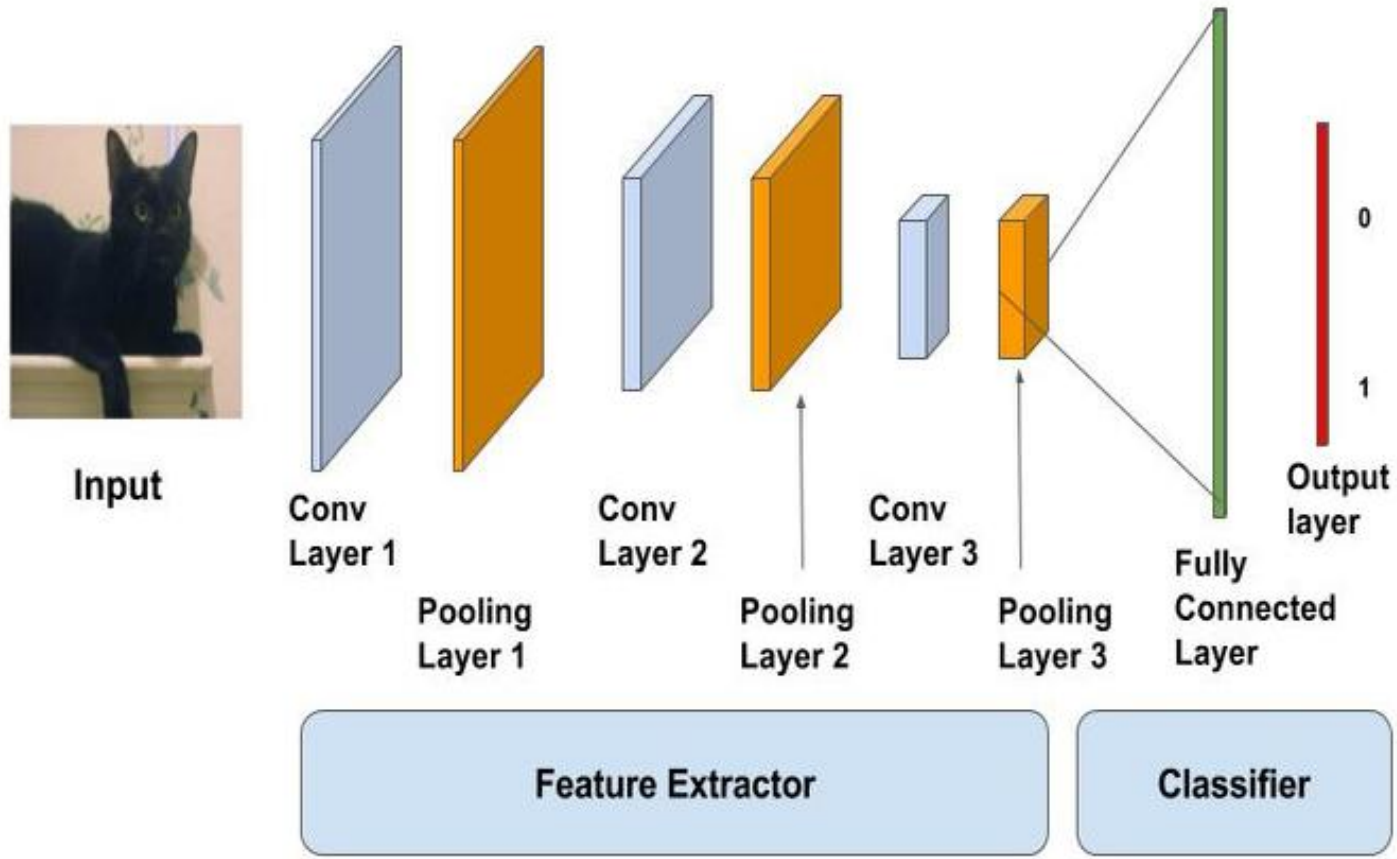


Image from `https://www.learnopencv.com/keras-tutorial-transfer-learning-using-pre-trained-models/`

Transfer learning

- More mathematically, `transfer learning` can be represented as follows.

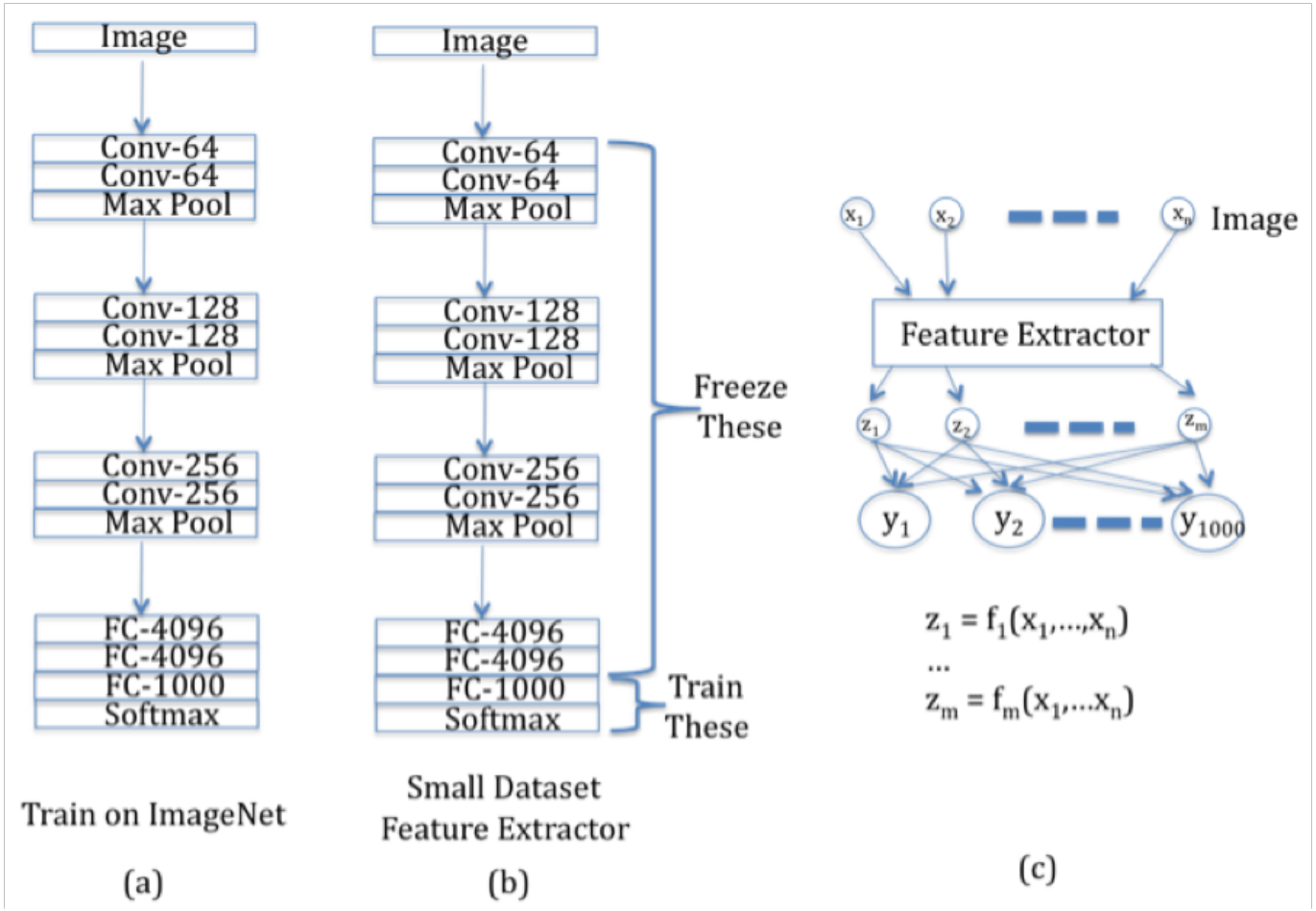


Image from `http://srdas.github.io/DLBook/ConvNets.html#transfer-learning`

Transfer learning

1. Strategy

- Take a `pre-trained CNN` (typically on `ImageNet`), e.g., `AlexNet`.
- Remove the last fully connected layer whose output are the 1000 different `class probabilities`.
- For all your new images, use the rest of the network as a feature extractor with `fixed weights`.

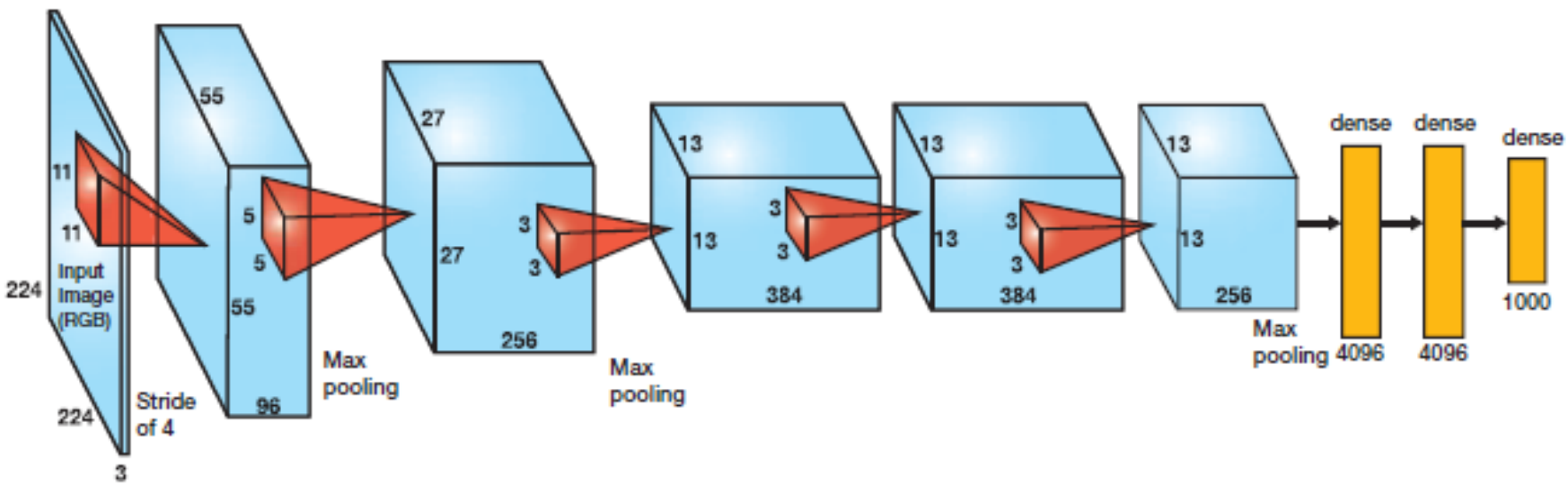


Image from `http://cv-tricks.com/cnn/understand-resnet-alexnet-vgg-inception/`

Transfer learning

1. Strategy

- Take a `pre-trained CNN` (typically on `ImageNet`), e.g., `AlexNet`.
- Remove the last fully connected layer whose output are the 1000 different `class probabilities`.
- For all your new images, use the rest of the network as a feature extractor with `fixed weights`.
- For each image you obtain a the activations/features (also called `codes`) of the last but one layer; for `AlexNet` a vector of dimension 4096.
- Train a `linear classifier` on the codes, e.g., `multinomial logistic regression` or `linear SVM`.

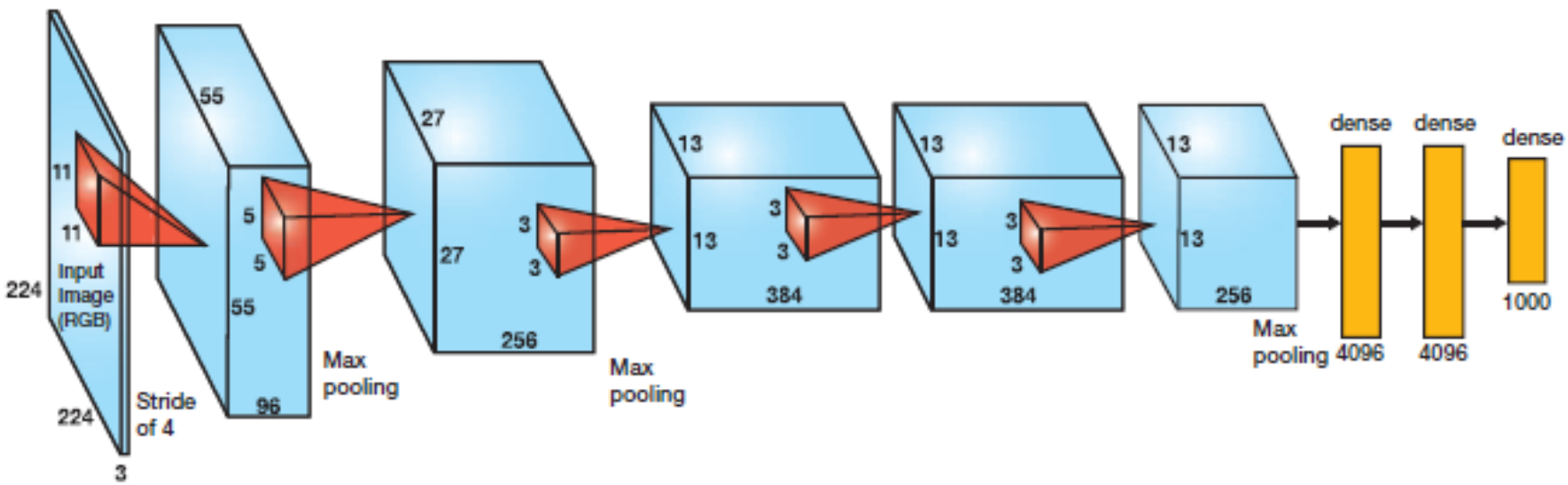


Image from `http://cv-tricks.com/cnn/understand-resnet-alexnet-vgg-inception/`

2. Strategy

- Alternatively, you can also `fine-tune` the CNN.
- Instead of only replacing and retraining the classifier, you can keep only some `early layers fixed` and fine-tune the later layers.
- This makes sense, since the early features of a CNN are generic such as `edge or blob detectors`, which are useful to many tasks.
- The later layers are more `specific` to the data set used during training.

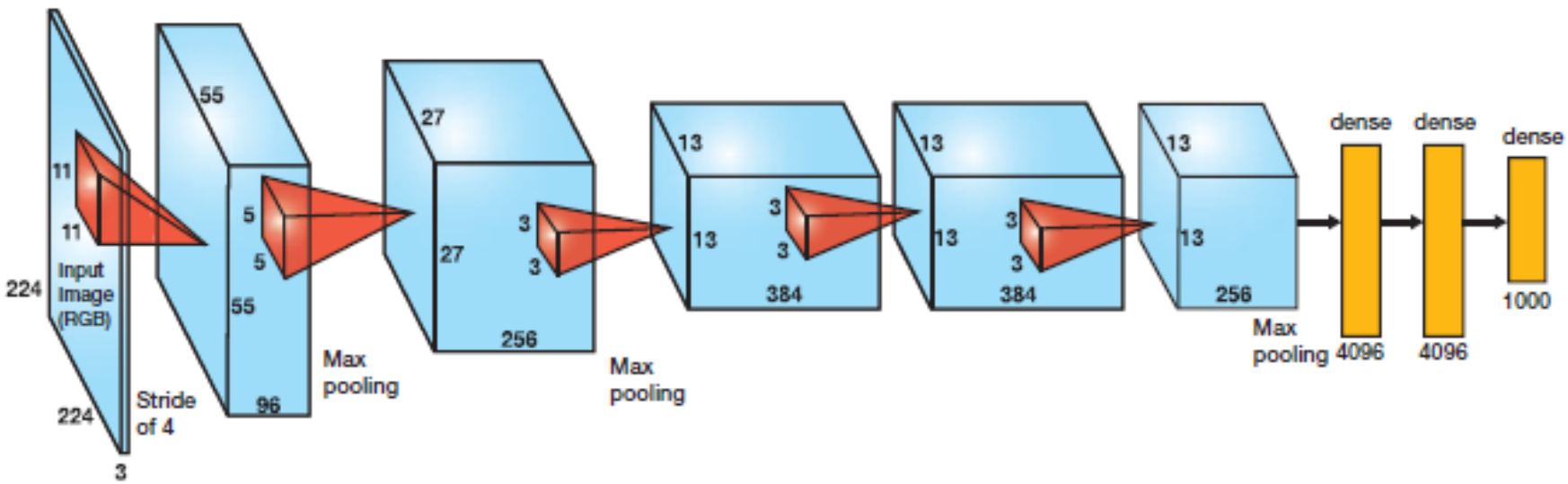


Image from `http://cv-tricks.com/cnn/understand-resnet-alexnet-vgg-inception/`

Transfer learning

- Which of the strategies you should choose depends on `different factors`: the `size` of your data set, the `similarity` to the original data set of the pre-trained model, etc.
- If your data set is `small` and `very similar` to the original data set, you should only train the classifier but not fine-tune the CNN due to `overfitting problems`.
- If your data set is `big` and `not too different` to the original data set, you can additionally fine-tune the CNN with less danger of overfitting.

Further topics

- Around "classical" deep neural networks there are plenty of new `applications/variants/extensions` that are published each month.
- For instance, `recurrent neural networks` can be used for times series forecasting.

Further topics

- Around "classical" deep neural networks there are plenty of new `applications/variants/extensions` that are published each month.
- For instance, `recurrent neural networks` can be used for times series forecasting.
- A popular topic at the moment are for instance `Generative Adversarial Networks` (GANs), see Goodfellow et al. (2014).

Further topics

- Around "classical" deep neural networks there are plenty of new `applications/variants/extensions` that are published each month.
- For instance, `recurrent neural networks` can be used for times series forecasting.
- A popular topic at the moment are for instance `Generative Adversarial Networks` (GANs), see Goodfellow et al. (2014).
- So-called `autoencoders` use deep neural networks to do dimension reduction and data compression.

Further topics

- Around "classical" deep neural networks there are plenty of new `applications/variants/extensions` that are published each month.
- For instance, `recurrent neural networks` can be used for times series forecasting.
- A popular topic at the moment are for instance `Generative Adversarial Networks` (GANs), see Goodfellow et al. (2014).
- So-called `autoencoders` use deep neural networks to do dimension reduction and data compression.
- Deep neural network are also used in `Reinforcement Learning`.

Reinforcement Learning

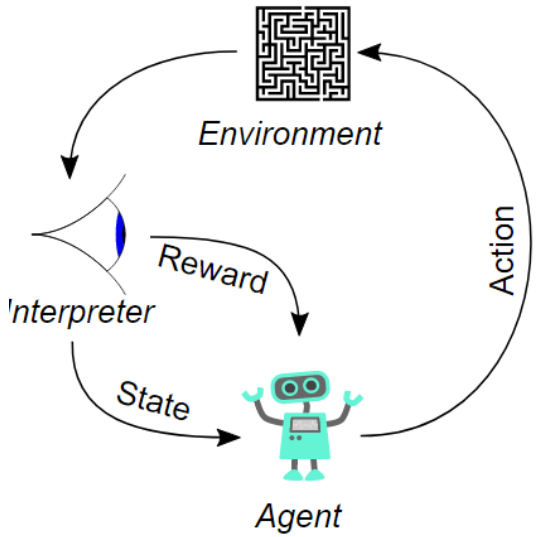


Image from https://en.wikipedia.org/wiki/Reinforcement_learning

- Reinforcement Learning involves an agent who takes actions in an environment; this action results in a reward and a change of the state; over many iteration the agent learns an optimal strategy of action to maximize the reward.
- In complex settings, this learning involves deep neural networks.
- "An Outsider's Tour of Reinforcement Learning": http://www.argmin.net/2018/06/25/outsider-rl/
- A nice Reinforcement Learning course by David Silver (AlphaGo) can be found on youtube.

Reinforcement Learning

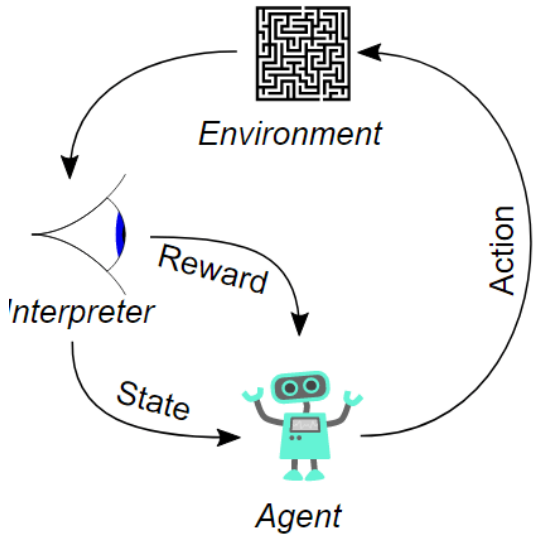


Image from https://en.wikipedia.org/wiki/Reinforcement_learning

- Reinforcement Learning involves an agent who takes actions in an environment; this action results in a reward and a change of the state; over many iteration the agent learns an optimal strategy of action to maximize the reward.
- In complex settings, this learning involves deep neural networks.
- "An Outsider's Tour of Reinforcement Learning": http://www.argmin.net/2018/06/25/outsider-rl/
- A nice Reinforcement Learning course by David Silver (AlphaGo) can be found on youtube.
- And to learn how computers defeated humans in the game Go, watch the documentary film AlphaGo.